

■ Projet 2 : "The Over-Engineered Guestbook"

Du Local au Cloud — CI/CD & Déploiement Production

Tu as conteneurisé ton application. Bravo. Maintenant, finis le travail : déploie-la sur un vrai serveur accessible depuis n'importe où dans le monde, avec un pipeline automatisé qui redéploie à chaque *git push*.

Durée estimée : **2 à 3 jours × ~3h**. Faisable, mais dense. Si tu bloques plus de 30 minutes sur un point, c'est normal — cherche, essaie, et reviens au guide.

■ Pourquoi c'est différent du local ?

Bonne question. Quand tu lances **docker-compose up** sur ton ordi, c'est parfait pour toi. Mais personne d'autre ne peut y accéder. Un déploiement cloud, c'est faire tourner exactement les mêmes conteneurs sur un **serveur distant** (une VM) qui, lui, a une adresse IP publique.

La conteneurisation rend ça **très simple** : le serveur n'a pas besoin de connaître Python, Flask, ou PostgreSQL. Il a juste besoin de **Docker**. Ton `docker-compose.yml` suffit à tout décrire. C'est tout l'intérêt.

Sans Docker	Avec Docker
Installer Python, pip, psycopg2, gunicorn... sur le serveur	Installer Docker sur le serveur, c'est tout
"Ça marche sur ma machine" ■	L'image est identique partout
Migrer = tout réinstaller	Migrer = docker-compose up

■■ L'architecture cible

Tu vas utiliser trois outils cloud que tu retrouveras dans 90% des projets pro :

- **GCP Compute Engine** (ou n'importe quel VPS) — Une VM Linux dans le cloud qui fait tourner Docker.
- **Docker Hub** — Un registre où stocker tes images Docker (comme GitHub pour le code).
- **GitHub Actions** — Un système de CI/CD : à chaque push sur main, il build l'image, la pousse sur Docker Hub, et redéploie sur ta VM.

■ Mots-clés à rechercher : VPS vs Cloud VM, Docker registry, CI/CD pipeline, SSH key authentication

■ Jour 1 — Le Serveur dans le Cloud

Objectif du jour

Avoir ta VM sur GCP avec Docker installé, et y déployer ton application **à la main** (sans automatisation). Si ça tourne sur le serveur, la base est solide.

1. Crée un compte GCP et lance une VM

Google offre 300\$ de crédits gratuits pour les nouveaux comptes. Tu vas créer une **instance Compute Engine** (une VM Linux).

- Va sur **console.cloud.google.com** → Compute Engine → Créer une instance.
- Choisis : région Europe, type **e2-micro** (gratuit/très peu cher), OS **Ubuntu 22.04 LTS**.
- Active l'option **"Autoriser le trafic HTTP"** et **"HTTPS"**.
- Note bien l'**adresse IP externe** de ta VM — c'est ton adresse publique.

■ Vidéo recommandée : cherche "GCP Compute Engine create VM tutorial 2024" sur YouTube (5-10 min).

2. Connecte-toi à ta VM en SSH

GCP propose un bouton SSH directement dans le navigateur. Utilise-le pour commencer. Plus tard, tu configureras une vraie clé SSH.

- Dans la console GCP, clique sur le bouton **SSH** à côté de ta VM.
- Une fenêtre de terminal s'ouvre dans le navigateur. Tu es maintenant *sur* ton serveur.

3. Installe Docker sur la VM

La VM est vierge. Tu dois y installer Docker (une seule fois). Cherche la procédure officielle pour Ubuntu :

```
# Recherche sur Google : "install docker ubuntu 22.04"  
# Suis la doc officielle : docs.docker.com
```

Installe aussi **docker-compose** (ou docker compose v2).

■ Vérifie que ça marche : **docker --version** et **docker compose version**

4. Transfère ton projet sur la VM

Deux approches possibles :

- **Option A (simple)** : git clone de ton repo GitHub directement sur la VM.
- **Option B (manuelle)** : scp (secure copy) depuis ton terminal local.

```
# Option A — depuis la VM :  
git clone https://github.com/TON_USERNAME/TON_REPO.git  
cd TON_REPO
```

5. Lance l'application sur la VM

Exactement comme en local. Ta VM a Docker, ton code est là :

```
docker compose up -d --build
```

Attends que tout démarre. Puis ouvre ton navigateur et va sur :

`http://[IP_EXTERNE_DE_TA_VM]`

■ Si ça ne s'affiche pas, vérifie les règles de pare-feu GCP : le port 80 doit être ouvert ("Firewall rules" dans la console).

Critère de réussite Jour 1 : Tu accèdes à ton livre d'or depuis ton téléphone, en utilisant l'IP publique de ta VM. ■

■ Jour 2 — CI/CD avec GitHub Actions

Objectif du jour

Automatiser le déploiement : quand tu pousses du code sur GitHub, le serveur se met à jour **tout seul**. C'est ce qu'on appelle un pipeline CI/CD (Continuous Integration / Continuous Deployment).

■ Vidéo recommandée : cherche "GitHub Actions deploy to server SSH tutorial" sur YouTube.

1. Crée un compte Docker Hub

Docker Hub est le registre public pour stocker tes images. Crée un compte gratuit sur **hub.docker.com**. Note ton **username** Docker Hub.

2. Configure les secrets GitHub

GitHub Actions a besoin de credentials pour se connecter à Docker Hub et à ta VM. Ces informations sensibles ne doivent **jamais** être dans ton code. GitHub propose les **Secrets** pour ça.

- Dans ton repo GitHub → Settings → Secrets and variables → Actions.
- Crée ces secrets : **DOCKERHUB_USERNAME**, **DOCKERHUB_TOKEN** (token API Docker Hub, pas ton mot de passe), **VM_HOST** (IP de ta VM), **VM_USER** (ubuntu), **VM_SSH_KEY** (ta clé privée SSH — voir ci-dessous).

■ Clé SSH : tu dois générer une paire de clés SSH (ssh-keygen), ajouter la clé publique dans ~/.ssh/authorized_keys sur ta VM, et mettre la clé PRIVÉE dans le secret GitHub. Cherche : "add SSH key to GitHub Actions secret".

3. Écris le workflow GitHub Actions

Crée le fichier **.github/workflows/deploy.yml** dans ton projet. Ce fichier décrit ce que GitHub doit faire automatiquement.

Ton workflow doit faire 3 choses :

- **Étape 1 (Build & Push)** : Builder l'image Docker de ton API et la pousser sur Docker Hub.
- **Étape 2 (Deploy)** : Se connecter en SSH à ta VM.
- **Étape 3 (Run)** : Faire un git pull + docker compose up --build -d sur la VM.

```
# Structure indicative du fichier :
name: Deploy
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Login to Docker Hub
        # utilise l'action docker/login-action
      - name: Build and push image
```

```
# utilise docker/build-push-action
- name: Deploy via SSH
  # utilise appleboy/ssh-action
```

■ Actions utiles à chercher dans GitHub Marketplace : *docker/login-action*, *docker/build-push-action*, *appleboy/ssh-action*

4. Teste le pipeline

Fais un petit changement dans ton code (ex: modifie un texte dans le frontend). Commit + push sur main. Va dans l'onglet **Actions** de ton repo GitHub et regarde le pipeline tourner en temps réel.

■ Si une étape échoue, GitHub Actions affiche les logs. Lis-les ! C'est là que tu trouveras l'erreur.

Critère de réussite Jour 2 : Tu modifies le code, tu push, et 2 minutes plus tard le changement est visible sur ton IP publique. Sans toucher au serveur. ■

■ Jour 3 — "Presque Production" — HTTPS & Variables d'environnement

Objectif du jour

Un vrai déploiement ne tourne pas sur une IP nue en HTTP. On va ajouter un **nom de domaine** (optionnel mais formateur), le **HTTPS**, et sécuriser les variables d'environnement.

1. Variables d'environnement en production

Ton docker-compose.yml a probablement des mots de passe en clair (POSTGRES_PASSWORD...). En production, on utilise un fichier **.env** qui n'est **jamais commité** sur Git.

- Assure-toi que **.env** est dans ton **.gitignore**.
- Sur ta VM, crée le fichier **.env** manuellement (une seule fois).
- Dans ton workflow GitHub Actions, tu peux aussi générer ce fichier à partir des secrets GitHub.

```
# Dans .github/workflows/deploy.yml :  
- name: Create .env file  
  run: |  
    echo "POSTGRES_PASSWORD=${{ secrets.DB_PASSWORD }}" >> .env  
    echo "SECRET_KEY=${{ secrets.SECRET_KEY }}" >> .env
```

2. HTTPS avec Let's Encrypt (Certbot)

HTTPS = connexion chiffrée. C'est indispensable dès qu'il y a des données. Let's Encrypt fournit des certificats SSL **gratuits**.

■ ■ Pour HTTPS, tu as besoin d'un vrai nom de domaine (ex: monapp.com). Tu peux en obtenir un gratuit sur freenom.com ou payer ~10€/an sur OVH/Namecheap. Sans domaine, cette étape est optionnelle.

- Installe Certbot sur ta VM.
- Modifie ta config Nginx pour gérer le domaine et le certificat.
- Certbot peut modifier la config Nginx automatiquement.

■ Cherche : "certbot nginx docker let's encrypt tutorial". Nginx Proxy Manager est une alternative plus simple si tu veux éviter la config manuelle.

3. Nom de domaine (optionnel mais recommandé)

Si tu as un domaine, configure un enregistrement **DNS de type A** qui pointe vers l'IP publique de ta VM. Cherche : "DNS A record configure domain".

4. Monitoring basique (bonus)

Comment savoir si ton app est tombée ? Quelques options simples :

- **UptimeRobot** (gratuit) : surveille une URL et t'envoie un email si elle ne répond plus.
- **docker compose logs -f** : voir les logs en temps réel sur la VM.
- **docker stats** : voir la consommation CPU/mémoire des conteneurs.

Critère de réussite Jour 3 : Ton app est accessible en HTTPS (ou en HTTP propre), les mots de passe ne sont pas dans le code, et tu as un monitoring basique. ■ ■

■ Boss Final — Critères de réussite du projet complet

Pour considérer ce projet terminé, tu dois pouvoir démontrer tout ceci :

1

L'app tourne en production

Accessible depuis n'importe quel navigateur, sur une vraie IP ou un domaine.

2

CI/CD fonctionnel

Un git push sur main déclenche un redéploiement automatique, visible dans l'onglet Actions.

3

Persistance

Les messages du livre d'or survivent à un docker compose down && docker compose up sur le serveur.

4

Pas de secrets dans le code

.env dans .gitignore, credentials dans GitHub Secrets.

5

Architecture réseau

Seul le port 80/443 est exposé publiquement. La BDD n'est pas accessible depuis internet.

■ Ressources et mots-clés pour tes recherches

- **GCP** : "GCP Compute Engine create VM free tier tutorial"
- **Docker Hub** : "push docker image to Docker Hub tutorial"
- **GitHub Actions** : "GitHub Actions deploy SSH VPS docker compose"
- **SSH Keys** : "generate SSH key pair Linux tutorial"
- **HTTPS** : "certbot nginx docker let's encrypt 2024"
- **Sécurité** : "docker compose environment variables secrets best practices"
- **CI/CD concept** : "CI/CD explained for beginners" (TechWorld with Nana sur YouTube)

■ Chaîne YouTube fortement recommandée : **TechWorld with Nana**. Ses vidéos sur Docker, CI/CD et Kubernetes sont les meilleures pour débiter.

■ Si tu arrives au bout de ce projet, tu auras une vraie expérience de déploiement cloud que beaucoup de développeurs junior n'ont pas. Tu pourras en parler en entretien avec des exemples concrets : "J'ai mis en place un pipeline CI/CD GitHub Actions qui déploie automatiquement une application Docker Compose sur GCP."